

STORY PROTOCOL · SECURITY REVIEW

Story Protocol Hook Review.

A security review of Hook Review.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Story Protocol	Hook Review	Guardian	November 3 – November 25, 2025	December 2, 2025

§ 00

Contents.

20 pages — overview, confidence ranking, scope, 11 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 10 findings	P. 08
	Remediation Review · 1 finding	P. 18
§ 05	Disclaimer	P. 19
§ 06	About Guardian	P. 20

§ 01

Overview.

Guardian reviewed Hook Review across 2 review rounds. The final report includes 2 Medium, 4 Low, 5 Informational findings.

AUDIT FIRM	REPOSITORY
Guardian	piplabs/protocol-core-v1-private-fork
CLIENT	METHODOLOGY
Story Protocol	Smart contract · Manual · Main Review · Remediation Review
PROJECT	REMEDIATION REVIEW COMMIT
Hook Review	acd7fb307761
PREPARED BY	
Guardian	
AUDIT WINDOW	
November 3 – November 25, 2025	
FINAL REPORT	
December 2, 2025	

1.1 Confidence Ranking

5 / 5

Very High Confidence

Guardian assigns a Confidence Ranking of 5 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
Critical	0	0	0	0	0
High	0	0	0	0	0
Medium	2	0	1	0	1
Low	4	0	2	0	2
Informational	5	0	1	0	4
All severities	11	0	4	0	7

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence ASSIGNED	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Hook Review in `piplabs/protocol-core-v1-private-fork`.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH

Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM

Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW

Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH

Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM

Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW

Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
No files were recorded in scope.		

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
M-01	maxMintingFee Broken For registerDerivative	Logical Error	MEDIUM	RESOLVED	p. 8
M-02	Whitelist Can Be Opened By EIP 7702	Access Control	MEDIUM	ACKNOWLEDGED	p. 9
L-01	calculateMintingFee Does Not Check Whitelist	Validation	LOW	RESOLVED	p. 10
L-02	Derivatives Registry Bypassing The Whitelist	Access Control	LOW	ACKNOWLEDGED	p. 11
L-03	Whitelist Does Not Prevent Transfers	Validation	LOW	ACKNOWLEDGED	p. 12
L-04	Inaccurate predictMintingLicenseFee Result	Logical Error	LOW	RESOLVED	p. 13
I-01	Redundant Code	Superfluous Code	INFORMATIONAL	RESOLVED	p. 14
I-02	Missing Zero Address Checks	Validation	INFORMATIONAL	RESOLVED	p. 15
I-03	Incorrect Whitelist Key Comment	Documentation	INFORMATIONAL	RESOLVED	p. 16
I-04	Whitelist Behavior On Owner Change	Warning	INFORMATIONAL	RESOLVED	p. 17
REMEDIATION REVIEW					
I-01	Inconsistent NatSpec	Documentation	INFORMATIONAL	ACKNOWLEDGED	p. 18

MEDIUM **M-01**

RESOLVED

maxMintingFee Broken For registerDerivative

CATEGORY Logical Error
LOCATION Out Of Scope
REVIEW Main Review

DESCRIPTION

In the `registerDerivative` function a `maxMintingFee` parameter exists to limit the maximum fees that a user must pay when invoking the function.

However the derivative registration can pull from multiple `licenseTermsIds` which may have differing fee currencies, making the shared maximum entirely ineffective for `licenseTermsIds` which would use several currencies which may have different decimals or wildly different prices.

RECOMMENDATION

Consider allowing the user to specify a list of `maxMintingFees` per `licenseTermsId` to match whatever unique currency each `licenseTermsId` could have.

RESOLUTION

Status: Resolved.

MEDIUM **M-02**

ACKNOWLEDGED

Whitelist Can Be Opened By EIP 7702

CATEGORY Access Control
LOCATION Global
REVIEW Main Review

DESCRIPTION

Story protocol supports EIP 7702 which allows the whitelist logic to be opened up to non-whitelisted callers in the `LicenseCallerWhitelistHook`.

EIP-7702 allows EOAs to delegate their execution capabilities to smart contracts. For example, if Bob is an approved caller, he can delegate his EOA to a simple proxy contract that exposes the `registerDerivative` and `mintLicenseTokens` functionality to any caller:

```
contract BobEOA {  
  LicensingModule licensingModule;  
  function registerDerivative(...) external {  
    licensingModule.registerDerivative(...);  
  }  
  function mintLicenseTokens(...) external {  
    licensingModule.mintLicenseTokens(...);  
  }  
}
```

The whitelist validation key is keyed off of the `ipOwner`, `licensorIpId`, `licenseTemplate`, `licenseTermsId`, and `minter` (caller), thus a caller who is whitelisted for some combination of those can open up the whitelist to any caller for that same combination.

RECOMMENDATION

Monitor whitelisted EOAs and consider removing them if they delegate to proxy contracts. Stay up to date with EIP-7702 developments, particularly proposals for delegate runtime introspection (discussion link) [<https://ethereum-magicians.org/t/transient-eip-7702-delegate-runtime-introspection-enabling-safer-smarter-wallet-innovation/24050>], which may allow safer detection and handling of such cases.

RESOLUTION

Status: Acknowledged.

LOW **L-01**

RESOLVED

calculateMintingFee Does Not Check Whitelist

CATEGORY Validation
LOCATION LicenseCallerWhitelistHook.sol
REVIEW Main Review

DESCRIPTION

The documentation for the `calculateMintingFee` function suggests that `The hook should revert if the minting fee calculation is not allowed`. However there is no whitelist validation performed, and thus the `calculateMintingFee` implementation does not abide by this documented behavior.

RECOMMENDATION

Consider if `_checkWhitelist` should be added to the beginning of the `calculateMintingFee` function, or if the documentation should be updated.

RESOLUTION

Status: Resolved.

LOW **L-02**

ACKNOWLEDGED

Derivatives Registry Bypassing The Whitelist

CATEGORY Access Control
LOCATION Global
REVIEW Main Review

— DESCRIPTION

In the `LicensingModule` there are two functions which allow for the registry of derivative child ip's, the first being `registerDerivative` and the second being `registerDerivativeWithLicenseTokens`.

`registerDerivative` will check the whitelist status by invoking the `beforeRegisterDerivative` function in it's flow. However the `registerDerivativeWithLicenseTokens` does not invoke the `beforeRegisterDerivative` function on the hook.

As a result, the whitelist status is not verified at the time of derivative registration in this second function. An account may have been whitelisted to initially mint these `licenseTokenIds`, however the whitelist status could have been revoked fro the account with the `removeFromWhitelist` function before the derivatives are registered with those tokens using the `registerDerivativeWithLicenseTokens` function.

Furthermore, the whitelist entries are invalidated every time the owner of the `licensorIpId` changes, the new owner may not expect anyone to be able to mint or register derivatives until they explicitly whitelist them. However as described, existing `licenseTokenIds` holders can still register derivatives.

— RECOMMENDATION

Consider introducing a new hook to be called at the time of `registerDerivativeWithLicenseTokens` invocation, or invoke the same `beforeRegisterDerivative` hook in this secondary derivative registration function.

— RESOLUTION

Status: Acknowledged.

LOW **L-03**

ACKNOWLEDGED

Whitelist Does Not Prevent Transfers

CATEGORY Validation
LOCATION Global
REVIEW Main Review

— DESCRIPTION

In case the LicenseToken is transferable the whitelist can be bypassed in the following way:

- Whitelisted user mints tokens
- Whitelisted user sells / transfers them to not whitelisted users

— RECOMMENDATION

Consider to prevent transfers of LicenseTokens if they use the whitelist hook or to add the whitelist hook to the transfer flow too.

Otherwise consider to document this behavior so that IP owners are aware.

— RESOLUTION

Status: Acknowledged.

LOW **L-04**

RESOLVED

Inaccurate predictMintingLicenseFee Result

CATEGORY	Logical Error
LOCATION	Out Of Scope
REVIEW	Main Review

DESCRIPTION

In the `_payMintingFee` flow if the `mintingFeeByHook` value is less than the `royaltyInfo.mintingFeeByLicense * amount` result then the transaction is reverted. However in the `predictMintingLicenseFee` function there is no accounting for such a minimum, and if the hook fee result is present it is always returned with no indication of if it would be less than the `mintingFeeByLicense` requirement leading to a revert.

RECOMMENDATION

Consider if the minimum `mintingFeeByLicense * amount` should be reflected in the `predictMintingLicenseFee` in some way. Either by a revert, a sentinel failure value, or reporting back the `mintingFeeByLicense * amount` value if it is greater than the hook fee.

RESOLUTION

Status: Resolved.

INFORMATIONAL **I-01**

RESOLVED

Redundant Code

CATEGORY	Superfluous Code
LOCATION	https://github.com/GuardianOrg/license-caller-whitelist-hook-team1-1761846435483/blob/main/contracts/LicenseCallerWhitelistHook.sol
REVIEW	Main Review

DESCRIPTION

The same two lines of code to calculate the whitelist key are written out four times in the contract:

```
address ip0wner = IIPAccount(payable(licensorIpId)).owner();
bytes32 key = keccak256(abi.encodePacked(ip0wner, licensorIpId, licenseTemplate,
licenseTermsId, minter));
```

RECOMMENDATION

Consider to move the code into a internal function.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-02

RESOLVED

Missing Zero Address Checks

CATEGORY	Validation
LOCATION	https://github.com/GuardianOrg/license-caller-whitelist-hook-team1-1761846435483/blob/7e7e9942efa734505ba05ebb226b37db9f023c5b/contracts/LicenseCallerWhitelistHook.sol#L62
REVIEW	Main Review

DESCRIPTION

The constructor of the `LicenseCallerWhitelistHook` does not perform a zero address check for the given `licenseRegistry`.

There are also no zero address checks for the parameters of the `addToWhitelist` function.

RECOMMENDATION

Consider to add a zero address checks to follow best practices.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-03

RESOLVED

Incorrect Whitelist Key Comment

CATEGORY Documentation
LOCATION LicenseCallerWhitelistHook.sol
REVIEW Main Review

DESCRIPTION

In the LicenseCallerWhitelistHook documentation for the whitelist mapping, the comment purports that the keys are based on: `keccak256(licensorIpId, licenseTemplate, licenseTermsId, minterAddress)`.

However this is not the case, instead the key is based on five entries instead of four, like so:

`keccak256(abi.encodePacked(ip0wner, licensorIpId, licenseTemplate, licenseTermsId, minter))`.

RECOMMENDATION

Correct the comment to reflect that the licensor `ip0wner` address is the first item in the key.

RESOLUTION

Status: Resolved.

INFORMATIONAL **I-04**

RESOLVED

Whitelist Behavior On Owner Change

CATEGORY Warning
LOCATION Global
REVIEW Main Review

— DESCRIPTION

If the owner of an IP changes than all whitelist entries are invalidated. But if the old owner receives the IP back later on the same whitelist entries are valid again.

This behavior may be unexpected for users.

— RECOMMENDATION

Be aware and consider to document this behavior.

— RESOLUTION

Status: Resolved.

INFORMATIONAL **I-01**

ACKNOWLEDGED

Inconsistent NatSpec

CATEGORY	Documentation
LOCATION	https://github.com/GuardianOrg/protocol-core-v1-private-fork-team1-1763750690972/blob/acd7fb307761325993febf0c32caa4e6e31bece/contracts/modules/licensing/LicensingModule.sol#L766-L788
REVIEW	Remediation Review

DESCRIPTION

The `_payMintingFee` does now return the `currencyToken` but does not include this return value in it's NatSpec.

RECOMMENDATION

Consider to add the `currencyToken` to the NatSpec.

RESOLUTION

Status: Acknowledged.

<https://github.com/piplabs/protocol-core-v1-private-fork/pull/103>

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.